DS 501 · INTRODUCTION TO DATA SCIENCE

Week 04

Databases and SQL

How relational databases organise data, how to query them with SQL for analysis, and when NoSQL stores, warehouses and lakes fit better.

THIS WEEK

4-1	**Relational Databases**	45 minutes
4-2	**SQL for Data Science**	45 minutes
4-3	**NoSQL and Data Repositories**	45 minutes
P4	**Practice: SQL on Chinook**	60 minutes · Python and SQL notebook

International Balkan University · [Instructor name] · [Semester]

Recap of Week 3

Three ideas from week 3 that this week builds on:

Where data comes from

Data is collected first-hand or reused from open portals, APIs and other organisations. Inside an organisation, most of it sits in **databases**.

Tidy data

Each variable is a column, each observation is a row, and each type of observational unit is a **table**. The third rule is the idea behind relational design.

Flat and nested formats

CSV stores flat rows; JSON stores **nested** records. Analysis usually needs a flat table, so nested data is flattened first.

This week: tidy tables become **relational** tables, and nested records become **documents**.

DS 501 · WEEK 4 · LESSON

4-1

Relational Databases

Lesson 1 of 3

In this lesson

01 Databases, DBMSs and Codd's relational model

02 Keys, relationships and the Chinook schema

03 Anomalies and normalisation to third normal form

04 Integrity constraints and ACID transactions

45 minutes

Builds on lesson 3-2 (tidy data)

Objectives

By the end of this lesson, you will be able to:

01 **Explain** what a DBMS does and the core ideas of Codd's relational model

Checked in: Quiz 4 · question 1

02 **Identify** primary, foreign, composite and surrogate keys, and the relationships they implement

Checked in: Quiz 4 · question 2

03 **Apply** the first three normal forms to remove redundancy and anomalies

Checked in: Quiz 4 · questions 3–4

04 **Describe** integrity constraints and the ACID properties of transactions

Checked in: Quiz 4 · question 5

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 03

Case: One Big Spreadsheet

THE SITUATION

A small online shop in Skopje keeps every order in one spreadsheet. Each row repeats the customer's name, address and phone number, and the product's name and price.

> “A customer moved. We changed her address on 3 of her 40 rows, and now our courier has two addresses for her.”

The owner also wants to list a new product that nobody has ordered yet.

YOUR TASK · PAIRS · 5 MIN

1. Why did the address problem happen? Name the design fault.

2. Why is it hard to record the new product in this sheet?

3. Sketch a better design. Which separate lists would you keep, and how would you link them?

Objective 01

Databases and Database Management Systems

- A **database** is an organised collection of data, stored so that it can be queried and updated efficiently.
- A **DBMS** is the software that stores the data, answers queries and enforces rules about valid data.
- It also controls who may see what, lets many users work at once, and recovers the data after a crash.
- A **relational** DBMS (RDBMS) stores data in tables and is queried with SQL.

Common relational systems

- SQLite: one file, no server
- PostgreSQL, MySQL, MariaDB
- Microsoft SQL Server
- Oracle Database, IBM Db2
- Managed in the cloud: Amazon RDS, Azure SQL, Cloud SQL

⚠ **Common mistake:** treating a spreadsheet as a database. A spreadsheet stores data, but it enforces almost no rules and breaks when several people edit it.

Codd's Relational Model

- In 1970, Edgar F. Codd of IBM proposed storing data as **relations**: tables of rows with named columns.
- Each column has a **domain**, the set of values it may hold, such as dates or positive numbers.
- Rows are identified by their **values**, through keys, not by their position in a file.
- Users state **what** data they want; the system decides how to find it.

Codd's terms

Relation = table

Tuple = row

Attribute = column

Domain = allowed values of a column

Schema = the tables, columns and keys

Key point: a relation is a **set** of rows, so rows have no built-in order. SQL results are unordered unless you ask for ORDER BY.

Anatomy of a Table: Chinook's Customer

The first four rows of the Customer table in the Chinook database (5 of its 13 columns shown):

CustomerId (PK)	FirstName	LastName	Country	SupportRepId (FK)
1	Luís	Gonçalves	Brazil	3
2	Leonie	Köhler	Germany	5
3	François	Tremblay	Canada	3
4	Bjørn	Hansen	Norway	4

In practice

Chinook has 11 tables. Customer has 59 rows, one per customer.

Read the last column. Agent 3 serves two customers here, but her name is stored once, in Employee.

Key point: a **primary key** identifies a row in its own table; a **foreign key** points to a row in another table.

Objective 02

Kinds of Keys

Key	What it is	Chinook example
Candidate key	Any minimal set of columns that identifies each row uniquely	CustomerId; also Email, if unique
Primary key	The candidate key chosen as the identifier: unique and never NULL	Customer.CustomerId
Foreign key	A column whose values must match a primary key in another table	Invoice.CustomerId
Composite key	A key made of two or more columns together	PlaylistTrack (PlaylistId, TrackId)
Surrogate key	A meaningless number generated by the system	TrackId, InvoiceId
Natural key	An identifier with business meaning	An ISBN or a national ID

</> In practice

Before trusting a key in pandas, test it: df["id"].is_unique.

Critical point: foreign keys are how tables link. Every join you write follows one.

Objective 02

Reading a Schema: Chinook

- Ten of Chinook's 11 tables are shown; MediaType is left out.
- PlaylistTrack is a **junction table**: it links playlists and tracks.

Objective 02

Types of Relationships

Relationship	Chinook example	How it is stored
One-to-one (1:1)	Not in Chinook; e.g. Employee and EmployeeContract	Foreign key with a UNIQUE rule, or a shared primary key
One-to-many (1:N)	One Customer has many Invoices	Foreign key on the “many” side: Invoice.CustomerId
Many-to-many (M:N)	Playlists contain many Tracks; Tracks are on many Playlists	A junction table with a composite key: PlaylistTrack
Self-referencing	Each Employee reports to another Employee	A foreign key to its own table: Employee.ReportsTo

Why? A many-to-many link always needs a third table. Counting through it counts pairs, not playlists.

⚠ **Common mistake:** putting the foreign key on the “one” side. A customer row cannot hold a list of invoice numbers; each invoice stores its customer.

Objective 03

Why Normalise? Three Anomalies

A table that stores several kinds of facts in one row suffers from three problems:

Update anomaly

One fact is stored on many rows.

Changing **some** of the rows leaves the data contradicting itself.

The customer with two addresses.

Insertion anomaly

A fact cannot be recorded until an unrelated fact exists.

A new product cannot be listed until someone **orders** it.

Deletion anomaly

Deleting one fact also deletes another.

Removing a customer's only order **erases** the customer.

Key point: **normalisation** stores each fact once, in the table of the entity it describes, and links tables with keys.

Objective 03

The First Three Normal Forms

Form	Rule	Typical violation	Fix
1NF	Every cell holds one atomic value; no lists or repeating groups	phones = “070 111 222, 071 333 444”	One row per phone in a Phone table
2NF	1NF, and every non-key column depends on the whole key	OrderLine [order_id, product_id], qty, product_name	Move product_name to Product
3NF	2NF, and non-key columns depend only on the key	Employee [emp_id], dept_id, dept_name	Move dept_name to Department

Note: 2NF matters only when the key is composite. Part of the key must not decide a column.

- Summary rule: every non-key column depends on **the key, the whole key and nothing but the key**.

⚠ **Common mistake:** confusing 2NF and 3NF. Ask whether the column depends on **part of a composite key** (2NF) or on **another non-key column** (3NF).

Objective 03

Exercise: Which Normal Form Is Violated?

Mark the lowest normal form each table violates and name the fix. Pairs, 4 minutes. Keys are in square brackets.

Table	1NF	2NF	3NF
Contact [contact_id], name, phones = “070 111 222, 071 333 444”			
Enrolment [student_id, course_id], grade, course_title			
Customer [customer_id], name, postal_code, city			
Track [track_id], name, genre_id, genre_name			
OrderLine [order_id, product_id], qty, unit_price_today			

Why? For each extra column, ask which column it really depends on.

Assume the postal code determines the city. One row is a trap: read the column's **meaning**, not its name.

Objective 03

Store each fact once. Join the facts when you need to read them.

Normalised tables keep the data consistent, because every change happens in one place. Analysis needs the facts side by side, so a data scientist rebuilds the wide table with joins, deliberately and only for reading.

Rule of thumb: if a correction must be made on more than one row, the design stores a fact twice.

Transactions and the ACID Properties

A **transaction** is a group of operations that the DBMS treats as one unit. Relational databases guarantee:

Atomicity

All operations in the transaction take effect, or none do. There is no half-finished state.

Consistency

Every transaction moves the database from one valid state to another; all constraints still hold.

Isolation

Concurrent transactions do not see each other's unfinished changes, as if they ran one after another.

Durability

Once a transaction is committed, its changes survive power failures and crashes.

In practice

A transfer of 100 is two updates: debit one account, credit the other. Atomicity ensures that a crash between them cannot make the 100 disappear.

Check Your Understanding

In Chinook, a playlist contains many tracks, and a track can appear on many playlists. How should this relationship be stored?

A Add a foreign key PlaylistId to the Track table.

B Store a comma-separated list of TrackIds in each Playlist row.

C Use a junction table with the composite key (PlaylistId, TrackId).

D Add a foreign key TrackId to the Playlist table.

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

DBMS
Software that stores data, answers queries and enforces rules.

Primary key
Columns whose values uniquely identify each row; never NULL.

Junction table
A table of key pairs that stores a many-to-many relationship.

Third normal form
Non-key columns depend on the key, the whole key, nothing else.

Relation
A table: a set of rows that share the same named columns.

Foreign key
A column whose values must match a primary key elsewhere.

Normalisation
Splitting tables so that each fact is stored exactly once.

ACID
Atomicity, consistency, isolation and durability of transactions.

Summary

In this lesson, you learned how to:

- **O1** Explain what a DBMS does and the core ideas of Codd's relational model
- **O2** Identify primary, foreign, composite and surrogate keys, and the relationships they implement
- **O3** Apply the first three normal forms to remove redundancy and anomalies
- **O4** Describe integrity constraints and the ACID properties of transactions

Key takeaway: store each fact once, link tables with keys, and let the DBMS keep the data valid.

Next lesson

4-2 SQL for Data Science

DS 501 · WEEK 4 · LESSON

4-2

SQL for Data Science

Lesson 2 of 3

In this lesson

01 What SQL is, and the order in which a query runs

02 Selecting, filtering, sorting and NULL values

03 Joining tables, and how joins multiply rows

04 Grouping, subqueries and window functions

45 minutes

Builds on lesson 4-1 (keys and the Chinook schema)

Objectives

By the end of this lesson, you will be able to:

01 **Explain** what SQL is and the logical order in which a query is evaluated

Checked in: Quiz 4 · question 6

02 **Apply** SELECT, WHERE and ORDER BY, and handle NULL values correctly

Checked in: Quiz 4 · question 7

03 **Apply** inner and left joins, and detect join fan-out

Checked in: Quiz 4 · question 8

04 **Apply** GROUP BY and HAVING to summarise groups

Checked in: Quiz 4 · questions 9 and 16

05 **Apply** subqueries, common table expressions and window functions

Checked in: Quiz 4 · question 10

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 03

Case: The Manager's Question

THE SITUATION

The sales manager of an online music store e-mails you on Monday morning:

> “Which genres earn us the most money, and who are our best customers in each country? Just export everything to Excel.”

The store's data sits in 11 tables in one database: the Chinook schema from lesson 4-1.

YOUR TASK · PAIRS · 5 MIN

1. Which tables hold the answer to each question? Trace the keys on the schema.
2. What could go wrong if you export the tables and match them by hand?
3. What would you send back to the manager?

Objective 01

What Is SQL?

- **SQL** (Structured Query Language) is the standard language of relational databases.
- It defines tables (CREATE, ALTER, DROP), changes rows (INSERT, UPDATE, DELETE) and reads them with **SELECT**.
- It is **declarative**: you describe the result, and the query optimiser decides how to compute it.
- Each product speaks a dialect. This lesson uses SQLite, as in Practice 4.

Why data scientists use it

- The data stays where it lives
- The database does the heavy work
- Python, R and BI tools all send SQL
- Analysts and engineers share it

Key point: a SELECT query only **reads** data. Data scientists spend most of their SQL time in SELECT.

The Logical Order of a Query

You write SELECT first, but the database evaluates a query in this logical order:

01

FROM and JOIN: choose the tables and combine them into one set of rows

02

WHERE: keep only the rows that meet a condition

03

GROUP BY: put rows with the same values into groups

04

HAVING: keep only the groups that meet a condition

05

SELECT: compute the output columns and their aliases

06

ORDER BY and LIMIT: sort the result and cut it to size

← **Why?** WHERE runs before GROUP BY, so it cannot filter on COUNT or SUM. That is the job of HAVING.

Selecting, Sorting and Limiting Rows

```sql
# SQL · Chinook in SQLite
SELECT Name, Composer,
       ROUND(Milliseconds / 60000.0, 1) AS minutes
FROM Track
ORDER BY Milliseconds DESC
LIMIT 10
```

1 Choose the columns to return.

2 · Critical: divide by **60000.0**. With 60000, SQLite divides integers and drops the decimals.

3 Read from the Track table.

4–5 Sort longest first and keep ten rows.

In practice

Result: the longest “track” is a TV episode, **Occupation / Precipice**, at 88.1 minutes. Outliers like this matter in any analysis of track length.

NULL: The Missing Value

- **NULL** means unknown or not applicable. It is not zero and not an empty string.
- Any comparison with NULL, even NULL = NULL, is **unknown**, and WHERE drops the row. Test with IS NULL.
- COUNT(*) counts rows; COUNT(column), SUM and AVG **skip** NULLs.
- COALESCE(Company, 'Private') replaces a NULL with a value.

Chinook's Customer table

COUNT(*) = **59**

COUNT(Company) = **10**

WHERE Company <> 'Google' returns **10** rows, not 59.

Why? The 49 customers with no company are neither equal nor unequal to 'Google'.

⚠ **Common mistake:** writing **WHERE Company = NULL**. It matches no rows, although 49 companies are missing, and no error warns you.

Inner Join Versus Left Join

Joining Employee to Customer on **SupportRepId = EmployeeId** in Chinook:

INNER JOIN

Keeps only rows that have a **match** in both tables.

Result: **59** rows, one per customer. Only the 3 sales support agents appear.

Use it when unmatched rows are irrelevant to the question.

LEFT JOIN

Keeps **every** row of the left table; missing matches become NULL.

Result: **64** rows: 59 matches plus the 5 employees with no customers.

Use it to find what is missing, such as the 1,519 tracks never sold.

Critical point: choose the join from the question: only matches, or everyone on the left side?

Objective 03

Joining Through the Schema: Revenue by Genre

```sql
# SQL · Chinook in SQLite
SELECT g.Name AS genre,
       ROUND(SUM(il.UnitPrice * il.Quantity), 2) AS revenue
FROM InvoiceLine il
JOIN Track t ON t.TrackId = il.TrackId
JOIN Genre g ON g.GenreId = t.GenreId
GROUP BY g.Name ORDER BY revenue DESC LIMIT 8
```

1–2 Revenue is price × quantity, summed per genre.

3 Start from the table that holds the money.

4–5 · Critical: each JOIN follows one foreign key on the schema path InvoiceLine → Track → Genre.

6 One row per genre, largest first.

In practice

Result: Rock **826.65**, Latin 382.14, Metal 261.36 (USD), out of 2,328.60 in total. Rock alone earns about 35%.

Join Fan-Out: When Totals Inflate

Total revenue in Chinook, computed twice as SUM(Invoice.Total):

Query	Rows	SUM(Total)
FROM Invoice	412	2,328.60
FROM Invoice JOIN InvoiceLine	2,240	20,848.62

In practice

Count rows before and after every join. If the count grows, check what you are summing.

Read the second row. Each invoice total repeats once per line, so the sum is almost nine times too large.

- Fix: sum Invoice.Total **before** joining, or sum UnitPrice × Quantity from InvoiceLine.
- The query runs without any error, which is what makes fan-out dangerous.

Objective 03

A join can multiply rows. Count them before and after.

Every one-to-many join repeats the rows on the “one” side. Sums and averages over those repeated values are wrong, yet the query runs without an error and the numbers look plausible.

⚠ Rule of thumb: know the grain of your result: what does one row represent after the join?

Objective 04

Summarising Groups with GROUP BY and HAVING

```sql
# SQL · Chinook in SQLite
SELECT c.Country, COUNT(DISTINCT c.CustomerId) AS customers,
       ROUND(AVG(i.Total), 2) AS avg_invoice
FROM Customer c JOIN Invoice i ON i.CustomerId = c.CustomerId
GROUP BY c.Country
HAVING COUNT(DISTINCT c.CustomerId) > 5
ORDER BY customers DESC
```

1–2 One value per country: customers and average invoice.

3 Join each invoice to its customer.

4 One group, and one output row, per country.

← **5 · Critical:** HAVING filters **groups** after aggregation. WHERE cannot see COUNT.

In practice

Result: only the **USA** (13 customers, average invoice 5.75) and **Canada** (8 customers, 5.43) have more than 5 customers.

Objective 05

Subqueries and Common Table Expressions

```
# SQL · Chinook in SQLite
1  WITH spend AS (
2    SELECT CustomerId, SUM(Total) AS spent
3    FROM Invoice GROUP BY CustomerId)
4  SELECT CustomerId, ROUND(spent, 2) AS spent
5  FROM spend
6  WHERE spent > (SELECT AVG(spent) FROM spend)
```

1–3 Name a first step: total spending per customer.

4–5 Read from it as if it were a table.

← **6 · Critical:** the subquery computes the average customer's spending, 39.47 USD. Only 22 of the 59 customers exceed it.

</> In practice

Practice 4 writes the same question with nested subqueries. The CTE version reads from top to bottom, like the cells of a notebook.

Objective 05

Window Functions: Ranking Within Groups

```sql
# SQL · Chinook in SQLite
SELECT c.Country, c.FirstName || ' ' || c.LastName AS customer,
       ROUND(SUM(i.Total), 2) AS spent,
       RANK() OVER (PARTITION BY c.Country
         ORDER BY SUM(i.Total) DESC) AS rank_in_country
FROM Customer c JOIN Invoice i ON i.CustomerId = c.CustomerId
GROUP BY c.CustomerId
```

1–2 One row per customer: name and spending.

← **3–4 · Critical:** PARTITION BY restarts the ranking in each country. Unlike GROUP BY, every row is kept.

5–6 Join invoices and total them per customer.

☆ **In practice**

Result for Brazil: Luís Gonçalves (39.62) ranks 1; four customers tie at 37.62 and all rank 2.

Objective 04

Check Your Understanding

You need the countries whose customers spent more than 100 in total. The query joins Customer and Invoice. Which clause is correct?

A WHERE SUM(i.Total) > 100, placed before GROUP BY c.Country

B WHERE i.Total > 100, placed before GROUP BY c.Country

C HAVING SUM(i.Total) > 100, placed after GROUP BY c.Country

D ORDER BY SUM(i.Total) DESC LIMIT 6, with no filter

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

SQL
The standard declarative language of relational databases.

Inner join
Combines rows of two tables that match on a condition.

Join fan-out
Rows repeated by a one-to-many join, inflating totals.

Common table expression
A named subquery defined with WITH.

NULL
A marker for a missing or unknown value; not zero or blank.

Left join
Keeps every row of the left table, with NULLs if unmatched.

HAVING
Filters groups after GROUP BY, using aggregate values.

Window function
Computes a value over related rows without collapsing them.

Summary

In this lesson, you learned how to:

- **O1** Explain what SQL is and the logical order in which a query is evaluated
- **O2** Apply SELECT, WHERE and ORDER BY, and handle NULL values correctly
- **O3** Apply inner and left joins, and detect join fan-out
- **O4** Apply GROUP BY and HAVING to summarise groups
- **O5** Apply subqueries, common table expressions and window functions

Key takeaway: trace the keys, know what one row means, then let the database filter, join and summarise for you.

Next lesson

4-3 NoSQL and Data Repositories

DS 501 · WEEK 4 · LESSON

4-3

NoSQL and Data Repositories

Lesson 3 of 3

In this lesson

01 What NoSQL means and why it emerged

02 Key-value, document, wide-column and graph databases

03 Scaling out, the CAP theorem, ACID and BASE

04 Warehouses, lakes and choosing a repository

45 minutes

Builds on lessons 4-1 and 4-2

Objectives

By the end of this lesson, you will be able to:

01 **Explain** what NoSQL means and why NoSQL databases emerged

Checked in: Quiz 4 · question 11

02 **Describe** key-value, document, wide-column and graph databases

Checked in: Quiz 4 · question 12

03 **Explain** scaling out, the CAP theorem, and ACID versus BASE

Checked in: Quiz 4 · question 13

04 **Distinguish** data warehouses, data marts, data lakes and lakehouses

Checked in: Quiz 4 · question 14

05 **Decide** which data repository fits a given workload

Checked in: Quiz 4 · question 15

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 05

Case: One Database for Everything?

THE SITUATION

A start-up in Skopje is building a food-delivery app. It must store user profiles with different fields, shopping carts that change every second, orders and payments, and which users follow which restaurants.

"Can we just put everything into one PostgreSQL database?"

YOUR TASK · PAIRS · 5 MIN

1. Describe the shape and access pattern of each kind of data.
2. Would one relational database work at launch? Why?
3. What might change with one million active users?

What Is NoSQL?

- **NoSQL**, usually read as “not only SQL”, is a family of databases that do not use relational tables as their main model.
- They emerged in the late 2000s, when web companies needed to serve millions of users from many cheap servers.
- Most have a **flexible schema**, scale out across machines, and relax some consistency guarantees.
- The boundary has blurred: many NoSQL systems now offer SQL-like queries and transactions.

Why they emerged

- Millions of users at once
- Data spread over many servers
- Records with varying fields
- Store objects as apps use them

Key point: NoSQL names what these systems are **not**, so it covers very different designs.

Objective 01

Relational Versus NoSQL Databases

Aspect	Relational (RDBMS)	NoSQL
Data model	Tables of rows and columns	Key-value, document, wide-column or graph
Schema	Defined before loading (schema-on-write)	Flexible; records can differ
Relationships	Foreign keys and joins	Embedded data, references or edges
Scaling	Traditionally up: a bigger server	Out: more servers
Consistency	Strong, with ACID transactions	Often relaxed or configurable
Query language	Standard SQL	Product-specific APIs and languages

Common mistake: reading this table as rules. PostgreSQL stores JSON, and some NoSQL systems support transactions.

Read this row first. NoSQL avoids joins by storing related data together.

Four Families of NoSQL Database

Key-value stores

Store a value under a unique key, like a huge dictionary. Very fast lookups. Examples: Redis, Amazon DynamoDB.

Document stores

Store self-contained documents, usually JSON, with nested fields. Examples: MongoDB, Couchbase.

Wide-column stores

Store rows in column families spread over many servers; built for huge write volumes. Examples: Apache Cassandra, HBase.

Graph databases

Store nodes and the relationships (edges) between them, and traverse them quickly. Examples: Neo4j, Amazon Neptune.

In practice

Design a NoSQL store around the **questions you will ask**; relational design starts from the structure of the data.

Objective 02

The Same Invoice as Rows and as a Document

```
# Invoice 1 as a JSON document (Practice 4)
1  {"invoice_id": 1, "date": "2021-01-01 00:00:00",
2   "customer_id": 2, "billing_country": "Germany", "total": 1.98,
3   "lines": [
4    {"track": "Balls to the Wall", "UnitPrice": 0.99, "Quantity": 1},
5    {"track": "Restless and Wild", "UnitPrice": 0.99, "Quantity": 1}]}
```

1–2 The invoice's own fields, from one Invoice row.

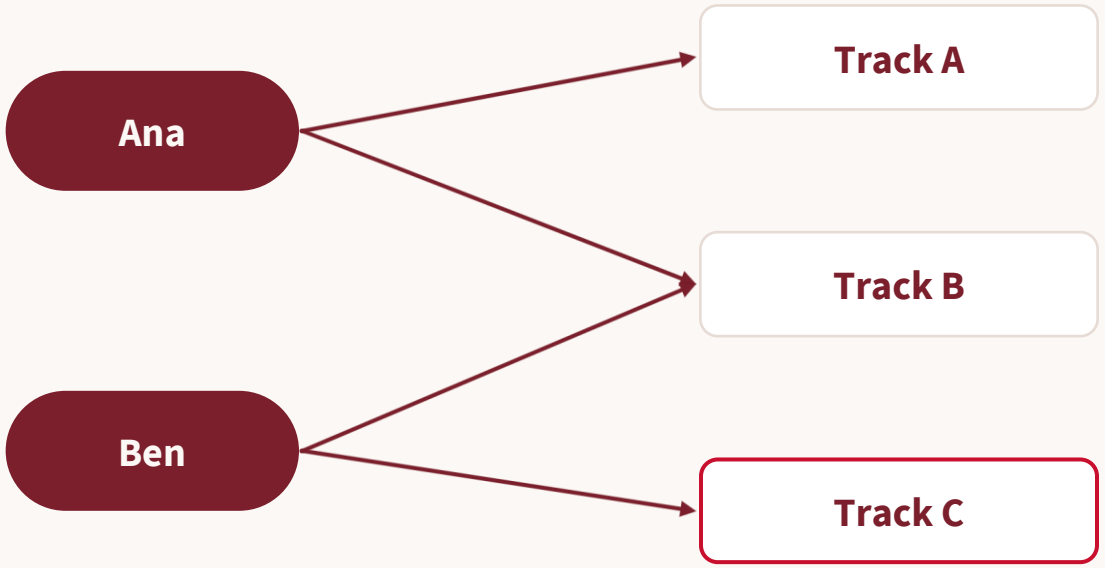
3–5 · Critical: the lines are **embedded**. One read returns the invoice; in Chinook it takes three tables.

In practice

Embed what is read together; reference what is shared and changes on its own, such as a track's name or a customer's address.

Objective 02

Graph Databases: Following Relationships




```
# Cypher (Neo4j) · illustrative
MATCH (a:Customer {name: 'Ana'})
  -[:BOUGHT]->(:Track)<-[:BOUGHT]-(:Customer)
  -[:BOUGHT]->(rec:Track)
RETURN rec
```

Critical point: Track C is two hops from Ana, via Ben. A graph database follows the edges directly.

Every arrow is a BOUGHT relationship (fictional data).

- In SQL, each hop is another self-join of the purchases table.
- In a graph database, each hop follows stored **edges**.

Scaling Out: Sharding and Replication

- **Scaling up** means a bigger server: simple, but limited and expensive at the top end.
- **Scaling out** means more servers, each holding part of the data and part of the work.
- **Sharding** splits the data by a key, such as a hash of the customer ID, across servers.
- **Replication** keeps copies on several servers, so data survives a failure and reads are shared.

Once data is spread out

- Servers talk over a network
- Networks can fail or split
- Copies can be briefly out of date
- The trade-off: the CAP theorem

In practice

Apache Spark in week 11 uses the same idea: split the data into partitions and process them on many machines at once.

The CAP Theorem

Brewer's conjecture (2000), proved by Gilbert and Lynch (2002): during a network **partition**, a distributed system must choose between C and A.

Consistency

Every read returns the **latest** write, whichever node answers.

Choosing C: refuse some requests rather than return stale data.

Availability

Every request to a working node receives a **response**.

Choosing A: answer every request, even if the answer may be out of date.

Partition tolerance

The system keeps running when the network **splits** nodes into groups.

Not optional: networks do fail, so P must be tolerated.

Critical point: CAP describes a partition. When the network is healthy, a system can be both consistent and available.

ACID Versus BASE

ACID

Atomicity, consistency, isolation, durability.

A committed change is immediately **correct** and visible to all.

Prefers consistency when the system must choose.

Needed for money, stock levels and bookings.

BASE

Basically available, soft state, eventually consistent.

Replicas may disagree briefly, then **converge**.

Prefers availability and low latency.

Acceptable for likes, view counts, feeds and caches.

⚠ **Common mistake:** assuming NoSQL means no transactions. MongoDB, for example, supports multi-document ACID transactions since version 4.0.

Repositories for Analysis

Analytical data is usually **copied** out of operational systems into repositories built for analysis:

Data warehouse

Cleaned, integrated history from many sources, organised for reporting, often as a star schema.

Data mart

A subset of the warehouse for one team or function, such as sales or finance.

Data lake

Raw data of any format in cheap storage; a schema is applied only when the data is read.

Lakehouse

A lake with warehouse features added: tables, transactions and governance on open files.

Key point: operational databases (**OLTP**) run the business; warehouses and lakes (**OLAP**) analyse it, from copies loaded by ETL.

Objective 05

Choose the store for the questions you will ask, not for the fashion.

Every repository makes some questions fast and others slow or impossible. Start from the data's shape, the access pattern and the cost of a wrong answer, then pick the simplest store that fits.

Rule of thumb: when in doubt, start with a well-supported relational database and add specialised stores when a real need appears.

Matching Workloads to Repositories

Workload	Good fit	Why
Orders, payments, account transfers	Relational database	Integrity and ACID transactions
Sessions and carts, read by session ID	Key-value store	Very fast reads by key
Product catalogue with varying attributes	Document store	Flexible schema; one read per item
Billions of sensor readings by device and time	Wide-column store	Massive write throughput
Accounts linked by shared phones and addresses	Graph database	Fast traversal of links
Company-wide reporting on history	Data warehouse	Integrated, fast analytics
Raw logs and files for future analysis	Data lake	Cheap storage, any format

Read the first row first. When an error costs money, ACID decides the choice before scale does.

In practice

Several rows have defensible alternatives. A good answer names the **property** that drove the choice.

Check Your Understanding

According to the CAP theorem, what must a distributed database give up while a network partition separates its nodes?

A Either consistency or durability: it cannot keep both.

B Either consistency or availability: it cannot keep both.

C Partition tolerance, because it has to pick any two of C, A and P.

D Sharding or replication: it must turn one of them off.

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

NoSQL
"Not only SQL": databases that do not use the relational model.

Graph database
Stores nodes and edges and traverses relationships quickly.

CAP theorem
During a partition, a system chooses consistency or availability.

Data warehouse
Integrated, cleaned historical data organised for analysis.

Document store
Stores self-contained, usually JSON, documents with nested fields.

Sharding
Splitting data by a key across several servers.

Eventual consistency
Replicas may differ briefly but converge without new updates.

Data lake
Raw data in any format; schema applied when it is read.

Summary

In this lesson, you learned how to:

- **O1** Explain what NoSQL means and why NoSQL databases emerged
- **O2** Describe key-value, document, wide-column and graph databases
- **O3** Explain scaling out, the CAP theorem, and ACID versus BASE
- **O4** Distinguish data warehouses, data marts, data lakes and lakehouses
- **O5** Decide which data repository fits a given workload

Key takeaway: every store makes some questions easy and others hard; choose by the questions, the shape of the data and the cost of a wrong answer.

Next

Practice 4 · SQL on Chinook

Practice 4: SQL on Chinook

You query a real relational database from Python, from simple filters to window functions.

Goal

Answer business questions with SQL, move the results into pandas, and interpret them.

Dataset

Chinook, a digital music store in one SQLite file: **11** tables, 59 customers, 412 invoices.

Notebook

The week 4 lab notebook, in Google Colab. The database downloads itself.

Time

60 minutes in class; finish the exercises at home before the week 5 session.

Parts A–C

Schema and filters; joins and aggregation; subqueries and window functions.

Then

SQL or pandas? And invoice 1 rebuilt as a JSON document, as a NoSQL store would keep it.

Exercises are marked ✏️. Trace the keys on the Chinook schema before you write each join.

Connecting and Reading the Schema

```python
# Python · Google Colab
DB = "chinook.sqlite"
con = sqlite3.connect(DB)
def q(sql, **params):
    return pd.read_sql_query(sql, con, params=params or None)
q("PRAGMA table_info(Invoice)")
q("PRAGMA foreign_key_list(InvoiceLine)")
```

1–2 Open the single-file database.

3–4 · Critical: every query comes back as a pandas **DataFrame**, ready for analysis.

5 Invoice: 9 columns; InvoiceId is the primary key.

6 InvoiceLine's foreign keys point to Track and Invoice.

In practice

Sketch the join path for revenue by genre and name the key used at each step.

Joining and Aggregating: Revenue by Country

```
# Python · Google Colab
revenue_by_country = q("""
SELECT c.Country, COUNT(DISTINCT c.CustomerId) AS customers,
       COUNT(i.InvoiceId) AS invoices,
       ROUND(SUM(i.Total), 2) AS revenue
FROM Customer c JOIN Invoice i ON i.CustomerId = c.CustomerId
GROUP BY c.Country ORDER BY revenue DESC""")
```

← **2 · Critical:** after the join, one row is an invoice. **DISTINCT** counts customers, not invoices.

3–4 Count invoices and add up their totals.

5 Join each invoice to its customer.

6 Result: USA 523.06, Canada 303.96, France 195.10.

In practice

🖍️ Revenue by genre with a bar chart, and the countries with more than 5 customers using HAVING.

Window Functions: A Running Total

```
# Python · Google Colab
monthly = q("""
SELECT strftime('%Y-%m', InvoiceDate) AS month,
       ROUND(SUM(Total), 2) AS revenue,
       ROUND(SUM(SUM(Total)) OVER (ORDER BY
       strftime('%Y-%m', InvoiceDate)), 2) AS running_total
FROM Invoice GROUP BY month ORDER BY month""")
```

2 Turn each date into a year-month label.

3 Revenue of each month.

← **4–5 · Critical:** the inner SUM totals each month; SUM(...) OVER adds the months up in order.

6 One row per month, in time order.

In practice

Result: 35.64, 73.26, 110.88 for January–March 2021, reaching **2,328.60** in December 2025. 🖍️ Plot it.

What to Submit

Submit the completed notebook (.ipynb) on the course page. It must run from top to bottom without errors.

Part	What is marked	Points
Run	The notebook runs from top to bottom (Restart and run all)	2
A	Join path sketched with keys; longest tracks in minutes	2
B	Revenue by genre with chart and share of total; HAVING query	3
C	Running total plotted and checked against the grand total	1
5–6	SQL or pandas answered; one pro and one con of documents	2
	Total	10

⚠ **Common mistake:** pasting a result table without a sentence that interprets it. Every number needs a sentence.

← **Due:** [date], before the week 5 session.

WEEK 4 · PAPERS TO READ

Papers to Read

The paper that founded relational databases, a survey of the NoSQL stores that challenged them, and two later reflections on what each side got right.

Required

Required Reading for Week 5

Read the listed sections before the week 5 class. One question in Quiz 5 may refer to these papers.

A Relational Model of Data for Large Shared Data Banks
Codd · Communications of the ACM 13(6), 1970

The founding paper of relational databases. Read section 1.

Scalable SQL and NoSQL Data Stores
Cattell · ACM SIGMOD Record 39(4), 2010

A survey of NoSQL designs. Read the introduction, one store type and the conclusions.

Bring a half-page note per required paper: the problem, the key idea, and its link to this week.

Optional

Optional Reading

CAP Twelve Years Later: How the “Rules” Have Changed

Brewer · IEEE Computer 45(2), 2012

The author of CAP corrects the “two of three” reading. About seven pages.

What Goes Around Comes Around… And Around…

Stonebraker & Pavlo · ACM SIGMOD Record 53(2), 2024

Fifty years of data models, and why SQL keeps absorbing them.

Optional papers are not examined. Choose one if your project touches distributed or non-relational data.

Required

Relations Protect Users From Storage Details

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

THE PROBLEM

In 1970, programs depended on how data was stored: record order, index chains, and tree or network access paths. When storage changed, programs broke. Codd asks how to shield users and programs from these internal details.

KEY IDEAS

01 Describe all data as relations: sets of rows with no meaningful row order and no duplicate rows

02 Primary keys identify rows; foreign keys refer to other relations by value, not by pointers

03 Normalization turns nested repeating groups into flat tables that carry the parent's key

Key term: Data independence — Programs and users keep working when the storage order, indexes or access paths of the data change

Required

Say What Data You Want, Not How to Reach It

WHAT TO TAKE AWAY

1. Names three dependencies to remove: on stored ordering, on indexes, and on access paths
2. Proposes a high-level data language based on predicate logic, the idea that later led to SQL
3. Defines projection, join and restriction, and uses them to reason about redundancy and consistency

THIS WEEK

Chinook's tables (Artist, Album, Track, Invoice) follow this model: keys link rows, and your SQL from Python never mentions file layout or indexes. This is the core of lesson 4-1.

LIMITATION

The paper is theory: it does not cover implementation or a full query language. SQL later departed from it, e.g. by allowing duplicate rows.

For your half-page note: Take one Chinook query. Which storage details (row order, indexes, file layout) does it not depend on, and why does that help?

Required

Comparing Scalable SQL and NoSQL Data Stores

Cattell, R. (2010). Scalable SQL and NoSQL data stores. ACM SIGMOD Record, 39(4), 12–27. https://doi.org/10.1145/1978915.1978919

THE PROBLEM

Web applications with millions of users need many small reads and writes per second. Traditional databases scaled poorly across many servers, and dozens of new data stores appeared. Cattell surveys and compares over twenty of them.

KEY IDEAS

01 Groups stores by data model: key-value, document, extensible record (wide column), scalable relational

02 NoSQL stores scale out by partitioning (sharding) and replicating data over many servers

03 Most give up full ACID transactions for weaker, often eventual, consistency

Key term: Horizontal scaling — Spreading data and load over many servers that share no RAM or disk, instead of using one bigger machine

Required

No Single Data Store Is Best for Every Use

WHAT TO TAKE AWAY

1. Key-value stores fit lookups by one key; document stores fit varied objects searched on many fields
2. Scalable SQL systems can compete if operations and transactions mostly stay on one node
3. Predicts NoSQL will last, new relational systems will grow, and products will consolidate

THIS WEEK

Gives lesson 4-3 its map of NoSQL store types. Compare the joins and transactions you use on Chinook with stores that offer only key lookups or single-record updates.

LIMITATION

A 2010 snapshot: many systems have since changed or disappeared, and the author notes that few independent benchmarks backed the scaling claims.

For your half-page note: For the Chinook music store, which data would you keep relational, and which might fit a key-value or document store? Why?

Optional

CAP Is a Trade-off to Manage, Not Pick Two of Three

Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. Computer, 45(2), 23–29. https://doi.org/10.1109/MC.2012.37

THE PROBLEM

The CAP theorem was often read as: choose two of consistency, availability and partition tolerance. Brewer, who first proposed CAP, argues this reading is misleading and explains how designers should handle network partitions.

KEY IDEAS

01 Partitions are rare, so without one a system need not give up consistency or availability

02 C and A are not on/off; the choice can differ per operation, per data item, or per subsystem

03 Handle partitions in three steps: detect the partition, enter partition mode, then recover

Key term: Network partition — A communication failure that splits a distributed system into groups of nodes that cannot reach each other

Optional

Plan How to Recover From Partitions

WHAT TO TAKE AWAY

1. A partition is a time limit on communication: on timeout, cancel (less A) or go on (risk to C)
2. During a partition, delay or limit risky operations, e.g. cap ATM withdrawals, and compensate later
3. Version vectors and CRDTs help replicas merge their state after a partition ends

THIS WEEK

Explains the consistency trade-offs behind the NoSQL stores in lesson 4-3. Our Chinook lab runs SQLite on one machine, so it never faces this choice.

LIMITATION

An essay with examples, not a measured study: it gives design advice but no formal proofs or performance results for its techniques.

To think about: An online music store's data centres lose contact. Should it keep selling? Which rule could break, and how would you fix it later?

Optional

SQL and the Relational Model Keep Absorbing Rivals

Stonebraker, M., & Pavlo, A. (2024). What goes around comes around... and around... ACM SIGMOD Record, 53(2), 21–37. https://doi.org/10.1145/3685980.3685984

THE PROBLEM

Since 2005, many systems tried to replace SQL or the relational model: MapReduce, key-value, document, column-family, text, array, vector and graph systems. The authors ask what happened to them over 20 years.

KEY IDEAS

01 Most non-relational systems stayed niche or are moving toward SQL and the relational model

02 Almost every major NoSQL system added a SQL-like language, and many added ACID transactions

03 The big changes were in how relational systems are built: column stores, cloud, data lakes

Key term: Lakehouse — An engine that runs SQL and DataFrame work over open-format files, such as Parquet, kept in cloud object storage

Optional

New Data Models Rarely Replace SQL; SQL Absorbs Them

WHAT TO TAKE AWAY

1. MapReduce and Hadoop faded; document stores and relational systems are converging
2. Relational systems added vector search fast, and SQL:2023 added property graph queries
3. Column storage now dominates analytics, and the cloud has reshaped how databases are built

THIS WEEK

Updates Cattell's 2010 picture for lesson 4-3 and supports lesson 4-2: SQL skills from the Chinook lab carry over to many modern systems. It also discusses pandas DataFrames.

LIMITATION

A strongly argued opinion paper by two relational-database researchers. Its predictions are judgements, not measured results.

To think about: Cattell (2010) predicted NoSQL would last. What do Stonebraker and Pavlo say happened by 2024, and do you agree?

Discussion

Reading Questions for Discussion

- Codd separated what data means from how it is stored. Why does this matter to a data scientist who only reads data?
- Cattell's stores give up joins and full ACID to scale out. Which of his store types, if any, would suit your project data?
- Brewer says “two of three” is misleading. Explain why, using a seat-booking system.
- Stonebraker and Pavlo say new data models drift back to SQL. Is that true of document stores?

Prepare for class

- Read the assigned sections
- Write a half-page note per paper
- Bring one answer to each question
- Discuss in groups of three

Key point: a good answer states a **position** and supports it with one specific passage from the paper.

DS 501 · WEEK 4

This Week's Work

Practice 4 notebook

Finish the week 4 lab notebook, check that it runs from top to bottom, and submit the .ipynb file.

Due: [date]

Quiz 4

15 multiple-choice questions on lessons 4-1 to 4-3, and one short applied SQL question on Chinook.

Due: [date]

Project step 4

Describe where your data is stored, its unit of analysis and its key. Check the key with df.duplicated(subset=key).sum().

Due: [date]

Next week: Data Cleaning, Wrangling and EDA: find and fix the errors in real data, reshape it with pandas, and explore it before modelling. Bring your project data: the data audit is due.